Practical Experiment Worksheet-5

SQL Table Management and Conditional Logic

---

1. Aim of the Experiment

To demonstrate the creation of a database table, insertion of records, and the use of the **CASE expression with the modulo function** in SQL to perform conditional analysis on salary values and determine whether the salary is even or odd.

---

2. Objectives of the Experiment

The objectives of this experiment are:

1. To understand how to create and manage database tables using SQL.
2. To learn how to remove existing tables using the **DROP TABLE** command.
3. To practice inserting records into a table using **INSERT statements**.
4. To understand the use of the **MOD() function** in SQL to find the remainder.
5. To implement conditional logic using the **CASE WHEN THEN ELSE expression**.
6. To retrieve and analyze data using **SELECT queries**.

---

3. Software / Tools Required

- Oracle Database / Oracle FreeSQL
- SQL Developer or SQL*Plus
- Computer system with database access

---

4. Theory

Structured Query Language (**SQL**) is used to manage and manipulate relational databases. SQL allows users to create database structures, insert records, retrieve information, and perform conditional operations on stored data.

In this experiment, a table named **employee** is created to store employee details such as employee ID, employee name, and salary.

Table Management

SQL provides **Data Definition Language (DDL)** commands to define database structures. Examples include:

- **CREATE TABLE** – Used to create a new table.
- **DROP TABLE** – Used to remove an existing table.

Data Manipulation

SQL also provides **Data Manipulation Language (DML)** commands to insert and manage data.

Example:

- **INSERT INTO** – Used to insert records into a table.

Data Querying

To retrieve data from the table, SQL uses **Data Query Language (DQL)**.

Example:

- **SELECT** – Used to fetch data from a table.

MOD Function

The **MOD() function** in SQL calculates the remainder after division.

Syntax:

MOD(number, divisor)

Example:

MOD(10,2) = 0
MOD(15,2) = 1

If the remainder is **0**, the number is even.
If the remainder is **1**, the number is odd.

CASE Expression

The **CASE expression** is used to implement conditional logic in SQL queries.

Syntax:

```
CASE
  WHEN condition THEN result
  ELSE result
END
```

In this experiment, the CASE expression is used to classify salaries as:

- **Even Salary**
- **Odd Salary**

---

5. Procedure

Step 1
Start the computer system and open the SQL environment such as Oracle SQL Developer or Oracle FreeSQL.

Step 2
Open a new SQL Query Editor window to write and execute SQL commands.

Step 3
Remove any previously existing **employee table** using the DROP TABLE command to ensure a clean environment.

Step 4
Create a new table named **employee** with the following fields:

- emp_id (Primary Key)
- emp_name
- salary

Step 5
Insert records into the table using **INSERT statements**.

Step 6
Write a **SELECT query** to retrieve employee details along with the remainder obtained when salary is divided by 2.

Step 7
Use the **CASE expression** to classify salaries as **Even Salary** or **Odd Salary**.

Step 8
Execute the query and verify the output.

Step 9
Record the output and take a screenshot for documentation.

---

6. SQL Commands Used in the Experiment

Step 1: Drop Existing Table

```
BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE employee';
EXCEPTION
  WHEN OTHERS THEN
    IF SQLCODE != -942 THEN
      RAISE;
    END IF;
END;
/
```

Step 2: Create Employee Table

```
CREATE TABLE employee (
   emp_id NUMBER PRIMARY KEY,
   emp_name VARCHAR2(50),
   salary NUMBER
);
```

Step 3: Insert Data into Table

```
INSERT INTO employee VALUES (101, 'Rahul', 50000);
INSERT INTO employee VALUES (102, 'Anita', 45001);
INSERT INTO employee VALUES (103, 'Vikram', 60000);
INSERT INTO employee VALUES (104, 'Priya', 55003);
INSERT INTO employee VALUES (105, 'Arjun', 70000);
INSERT INTO employee VALUES (106, 'Neha', 48007);

COMMIT;
```

Step 4: Query with Conditional Logic

```
SELECT
   emp_id,
   emp_name,
   salary,
   MOD(salary,2) AS remainder,
   CASE
      WHEN MOD(salary,2)=0 THEN 'Even Salary'
      ELSE 'Odd Salary'
   END AS salary_type
FROM employee;
```

---

7. Output

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.1 seconds

	EMP_ID	EMP_NAME	SALARY	REMAINDER	SALARY_TYPE
1	101	Rahul	50000	0	Even Salary
2	102	Anita	45001	1	Odd Salary
3	103	Vikram	60000	0	Even Salary
4	104	Priya	55003	1	Odd Salary
5	105	Arjun	70000	0	Even Salary
6	106	Neha	48007	1	Odd Salary

---

8. Observations

1. The table was successfully created using the CREATE TABLE command.
2. Six records were inserted into the employee table.
3. The MOD function correctly calculated the remainder of salary divided by 2.
4. The CASE expression successfully categorized salaries into **Even Salary** and **Odd Salary**.
5. The SELECT query retrieved the desired output with a new calculated column.

---

9. Result

The experiment was successfully performed.
The **employee table was created**, records were inserted, and SQL conditional logic using **MOD() and CASE expressions** was applied to classify salaries as **Even or Odd**.